

Algoritmo de Duan

Resumo

O algoritmo de Duan (2025) representa um avanço relevante na área de algoritmos de grafos, sendo o primeiro algoritmo determinístico para o problema do caminho mínimo de fonte única (SSSP – Single-Source Shortest Path) a atingir complexidade sublogarítmica no fator associado à ordenação de vértices.

O resultado teórico obtido é dado por:

$$O(m \log^{\frac{2}{3}} n)$$

superando a barreira clássica imposta pelo algoritmo de Dijkstra, cuja complexidade é:

$$O(m + n \log n)$$

A principal contribuição do algoritmo consiste na substituição da ordenação global de vértices por uma estrutura hierárquica baseada em redução progressiva de fronteira, recursão controlada e relaxamentos locais em múltiplas escalas.

Parte 1: Limitações do Algoritmo de Dijkstra

O algoritmo de Dijkstra resolve o problema de caminhos mínimos de fonte única utilizando uma estrutura de prioridade para armazenar a chamada “fronteira ativa” de vértices, isto é, o conjunto de vértices candidatos à próxima expansão do caminho mínimo.

Essa fronteira, frequentemente denotada por S , pode atingir tamanho $\Theta(n)$ em grafos esparsos, exigindo operações contínuas de inserção e extração em uma fila de prioridade.

Tais operações possuem custo $\Theta(\log n)$, resultando em uma complexidade total dominada por:

$$\Theta(n \log n)$$

Esse custo decorre essencialmente da necessidade de manutenção de uma ordenação global dos vértices, mesmo quando apenas uma pequena fração deles é relevante em cada etapa da execução.

A ideia central do algoritmo de Duan consiste em remover essa dependência de ordenação global, substituindo-a por um controle local e hierárquico da exploração do grafo.

Parte 2: Parametros Fundamentais

O algoritmo introduz dois parâmetros fundamentais responsáveis por controlar o equilíbrio (trade-off) entre os dois paradigmas de execução empregados: a abordagem gulosa do algoritmo de Dijkstra e a estratégia iterativa do algoritmo de Bellman-Ford. Esses parâmetros são definidos como:

$$k = \lfloor \log^{1/3}(n) \rfloor$$

$$t = \lfloor \log^{2/3}(n) \rfloor$$

O parâmetro k determina a quantidade de iterações do algoritmo de Bellman-Ford realizadas com o objetivo de “completar” os vértices próximos, isto é, propagar informações de distância para regiões locais do grafo sem a necessidade de sucessivas operações de extração mínima da fila de prioridade. Dessa forma, reduz-se a dependência excessiva do mecanismo tradicional utilizado no algoritmo de Dijkstra.

Por sua vez, o parâmetro t controla o fator de divisão aplicado em cada nível do processo recursivo do algoritmo. Esse mecanismo organiza o problema em subestruturas menores, permitindo um balanceamento mais eficiente entre exploração local e global do grafo.

Como consequência dessa parametrização, a profundidade da recursão é limitada por:

$$\frac{\log^t(n)}{t} = O(\log^{1/3}(n))$$

Esse resultado implica que o número de níveis recursivos cresce de forma sublogarítmica, contribuindo diretamente para a melhoria da complexidade assintótica do algoritmo.

Parte 3: O Subproblema Central — BMSSP

O funcionamento do algoritmo principal depende diretamente de um subprocedimento denominado BMSSP (Bounded Multi-Source Shortest Path), responsável por calcular caminhos mínimos de maneira limitada e controlada dentro de determinadas regiões do grafo. Esse subproblema constitui o núcleo da proposta do algoritmo, permitindo reduzir o tamanho da fronteira processada e, conseqüentemente, minimizar os custos associados às operações tradicionais de prioridade.

O procedimento BMSSP recebe três parâmetros principais:

Parâmetro	Significado
1	Nível atual da recursão

Parâmetro	Significado
B	Limite superior de distância considerado relevante para a execução atual
S	Conjunto de vértices pertencentes à fronteira

O parâmetro l define a profundidade recursiva em que o algoritmo se encontra, influenciando diretamente o tamanho máximo permitido para o conjunto de fronteira. Já o valor B estabelece um limite superior de distância, restringindo a exploração apenas aos vértices cujo custo potencial seja considerado relevante naquele estágio do processamento. O conjunto S , por sua vez, representa os vértices de fronteira utilizados como múltiplas fontes para a expansão local do cálculo dos caminhos mínimos.

Ao término de sua execução, o procedimento retorna dois resultados fundamentais:

Retorno	Significado
$B' \leq B$	Novo limite de distância, representando uma fronteira refinada para a continuação do processamento
U	Conjunto de vértices completos, isto é, vértices cuja distância mínima $d(v)$ foi corretamente determinada e satisfaz $d(v) < B'$

Em termos práticos, o conjunto U contém os vértices considerados “finalizados”, cujas distâncias mínimas já foram calculadas com exatidão. O novo limite B' atua como uma fronteira refinada, permitindo que as próximas etapas do algoritmo concentrem esforços apenas em regiões relevantes do grafo, evitando expansões excessivas e reduzindo o custo computacional global. Essa estratégia é essencial para alcançar a melhoria de desempenho proposta pelo algoritmo determinístico de Duan.

Parte 4: FindPivots (Redução de Fronteira)

O algoritmo FindPivots constitui o núcleo do procedimento BMSSP e representa um dos principais mecanismos responsáveis pelo ganho de eficiência da abordagem proposta. Seu objetivo central é reduzir o conjunto de fronteira S a um subconjunto significativamente menor de vértices estratégicos, denominados pivôs P . Essa redução é fundamental para diminuir o custo computacional associado ao processamento dos vértices candidatos durante a execução do algoritmo, reduzindo a dependência de operações dispendiosas de ordenação e seleção.

Intuição do procedimento

O procedimento inicia-se considerando o conjunto:

$$W = S$$

e realiza k iterações de relaxamento local no estilo Bellman-Ford. Durante essas iterações, a informação de distância é propagada a partir dos vértices de W , gerando novos vértices alcançados.

A cada iteração: - novos vértices são adicionados ao conjunto W - a propagação é restringida pelo limite B - o crescimento de W é monitorado

Caso o conjunto W cresça além de um limite proporcional a $k \cdot |S|$, o algoritmo considera que a fronteira está excessivamente densa e retorna imediatamente $P = S$, evitando processamento adicional.

Caso contrário, é construída uma floresta de caminhos mínimos induzida por W , e os vértices de S cujas subárvores possuem tamanho suficientemente grande são selecionados como pivôs.

Pseudocódigo

Algoritmo FindPivots($B, S, db, graph, k$)

Início

```
W ← S
W_prev ← S

Para i ← 1 até k faça

    W_next ← ∅

    Para cada u ∈ W_prev faça
        Para cada (u, v) ∈ E faça

            Se db[u] + w(u,v) ≤ db[v] e db[u] + w(u,v) < B então
                db[v] ← db[u] + w(u,v)
                W_next ← W_next ∪ {v}
            FimSe

        FimPara
    FimPara

W ← W ∪ W_next
W_prev ← W_next
```

```

    Se  $|W| > k \cdot |S|$  então
        Retorne  $(S, W)$ 
    FimSe

FimPara

Construir floresta de caminhos  $F$  induzida por  $W$ 

 $P \leftarrow \emptyset$ 

Para cada  $u \in S$  faça
    Se  $\text{tamanho\_sub\acute{a}rvore}(u, F) \geq k$  então
         $P \leftarrow P \cup \{u\}$ 
    FimSe
FimPara

Retorne  $(P, W)$ 

Fim

```

Parte 5: Estrutura BMSSP

O BMSSP (Bounded Multi-Source Shortest Path) é um algoritmo que utiliza FindPivots para reduzir progressivamente a fronteira e executa chamadas recursivas em níveis inferiores.

A cada nível l , o problema é dividido em subproblemas menores, com parâmetros de distância reduzidos e conjuntos de vértices mais estruturados.

Ideia operacional

1. **Redução da fronteira via pivôs:** Utiliza o procedimento FindPivots para reduzir o conjunto de fronteira S a um subconjunto menor de pivôs P .
2. **Inserção em estrutura auxiliar de controle:** Os pivôs são inseridos em uma estrutura auxiliar que permite o controle do processamento.
3. **Processamento por blocos de distância:** O processamento é realizado por blocos de distância, permitindo um controle mais eficiente do algoritmo.
4. **Chamadas recursivas em nível inferior:** O algoritmo executa chamadas recursivas em níveis inferiores para processar subproblemas menores.
5. **Relaxamento incremental de arestas:** O relaxamento das arestas é realizado de forma incremental, permitindo um controle mais eficiente do algoritmo.

Pseudocódigo Simplificado

```
Algoritmo BMSSP( $l$ ,  $B$ ,  $S$ )

Se  $l = 0$  então
    Executa Dijkstra limitado em  $S$ 
    Retorna ( $B'$ ,  $U$ )
FimSe

( $P$ ,  $W$ )  $\leftarrow$  FindPivots( $B$ ,  $S$ ,  $db$ ,  $graph$ ,  $k$ )

Inserir  $P$  em estrutura  $D$ 

 $U \leftarrow \emptyset$ 

Enquanto houver blocos em  $D$  faça

    ( $B_i$ ,  $S_i$ )  $\leftarrow$   $D.extract()$ 

    ( $B'_i$ ,  $U_i$ )  $\leftarrow$  BMSSP( $l-1$ ,  $B_i$ ,  $S_i$ )

     $U \leftarrow U \cup U_i$ 

    Para cada  $u \in U_i$  faça
        Relaxar arestas de  $u$ 
    FimPara

FimEnquanto

Retorna ( $B'$ ,  $U$ )
Fim
```

Parte 6: Estrutura Geral do Algoritmo

O algoritmo de Duan pode ser interpretado como uma combinação estruturada de três ideias principais:

- execução local tipo Dijkstra em níveis iniciais
- propagação controlada tipo Bellman-Ford em janelas limitadas
- decomposição recursiva da fronteira por pivôs

Essa combinação permite eliminar a necessidade de ordenação global completa dos vértices, substituindo-a por um processamento hierárquico da estrutura do grafo.

Conclusão

O algoritmo de Duan introduz uma mudança estrutural no paradigma clássico de caminhos mínimos, ao substituir a ordenação global por um modelo baseado em:

- redução dinâmica de fronteira
- seleção de vértices pivôs
- recursão multi-nível
- relaxamento local controlado

Essa abordagem resulta em uma complexidade assintótica de:

$$O(m \log^{2/3}(n))$$

representando um avanço teórico significativo na literatura de algoritmos de grafos.